Estimating the Derivatives

Zhengxiao Wei, Farouk S. Nathoo, Eleah Stringer

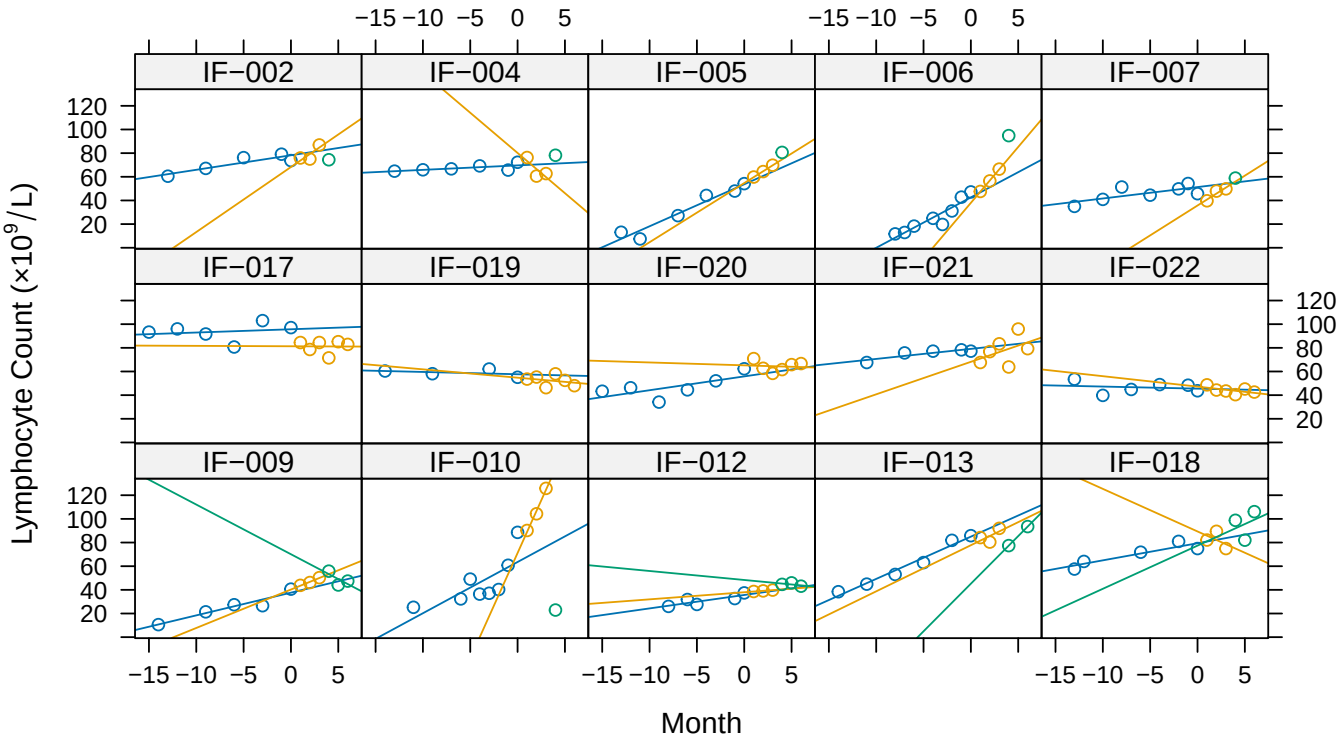
August 1 - September 11, 2023 (v2)

1. Data description (same as IF v3.5.Rmd p. 1)

The longitudinal data are from an experiment to study the effect of intermittent fasting (IF) on the lymphocyte measurements of 15 participants, each of whom was repeatedly measured before, during, and after the intervention. The data set (long format) consists of six columns:

- **LYMP** - [numeric] lymphocyte count ($\times 10^9$/L); lower values usually correspond to better outcomes;
- **ATG** - [numeric] relative net LC3-II expression; higher levels are associated with increased autophagy;
- **ID** - [factor] participant identifier; 15 levels;
- **Time** - [numeric] centered time point, with a frequency of 30 days, treating the **last** pretest as Month 0;
- **Time_Within_Phase** - [numeric] time point within each phase, starting from Month 0;
- **Phase** - [numeric] treatment identifier; 0 as pretest, 1 as intervention, and 2 as posttest.

```
df <- read.csv("~/Downloads/IF_v3.1.csv", stringsAsFactors=T)
lattice::xyplot(LYMP ~ Time | ID, data=df, group=Phase, type=c("p","r"), layout=c(5,3),
                index.cond=list(c(6:9,11,10,12:15,1:5)), #change panel order
                xlab="Month", ylab=expression(paste("Lymphocyte Count (\u00d7",10^9/L,")")))
```



2. Modeling

The hypothesis is that IF can help patients lower their lymphocyte count. The objectives are then to fit a curve for each subject's measurement and estimate the first and second derivatives of the function at Month 0 (representing the rate of change and the rate of change of the rate of change).

- First Derivative

Positive derivatives indicate the function is increasing, and negative derivatives indicate the function is decreasing.

- Second Derivative

If the second derivative is positive at a point, the curve is concave upward (e.g., ∪-shape); if the second derivative is negative, the curve is concave downward (e.g., ∩-shape).

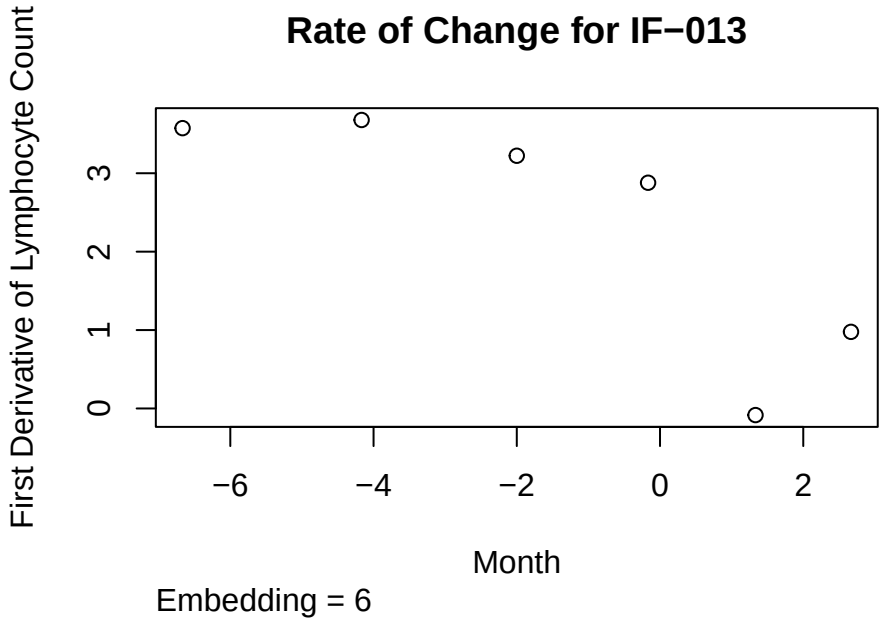
A point is a local minimum where the first derivative is zero and the second derivative is positive. Similarly, if the first derivative is zero and the second derivative is negative, that point is a local maximum. If the second derivative is zero and changes sign at a point, that point is an **inflection point** where the curve changes concavity.

2.1 Using the "*doremi*" package (v 1.0.0)

The *calculate.gold* function calculates the nth derivative at some measurement points (called `embeddings`) using the generalized orthogonal local derivative (GOLD) method, which is applicable to irregularly spaced data.

```
dat <- df[!is.na(df$LYMP),] #IF-013 contains one missing value
s13 <- subset(dat, ID=="IF-013")

der_est <- doremi::calculate.gold(time=s13$Time, signal=s13$LYMP, embedding=6, n=2)
plot(der_est$dtime, der_est$dsignal[,2], main="Rate of Change for IF-013",
     xlab="Month", ylab="First Derivative of Lymphocyte Count")
mtext("Embedding = 6", side=1, adj=0, line=4)
```



2.2 Using the default "*stats*" package (v 4.3.1)

The *smooth.spline* function involves fitting a cubic smoothing spline to the data and then evaluating the derivative of the spline. **Note** that the fitted curve will be different from the GOLD one in Section 2.1 (and 2.3), but the same as the *calculate.fda* function.

```
spline_fit <- smooth.spline(s13$Time, s13$LYMP) #fit a spline
der_est_1 <- predict(spline_fit, deriv=1)$y #compute the first derivative
# der_est <- doremi::calculate.fda(time=s13$Time, signal=s13$LYMP)$dsignal[,2] #same

names(der_est_1) <- s13$Time
der_est_1
```

```
##      -14      -11       -8       -5       -2        0        1        2
## 3.126697 3.161885 3.214658 3.139196 2.761960 2.390951 2.230657 2.119399
##        3        4        6
## 2.050824 2.030158 2.042499
```

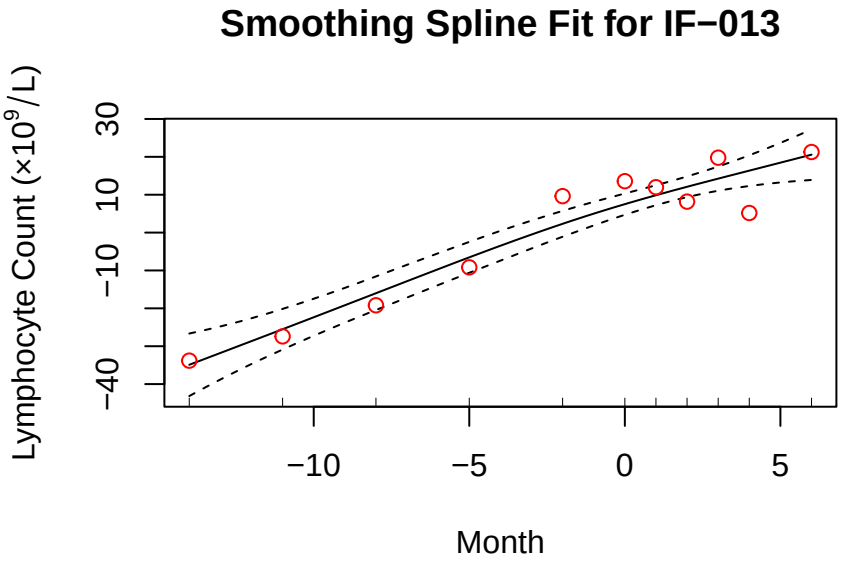
2.3 Using the default "*mgcv*" package (v 1.9-0)

The *gam* function fits generalized additive models (GAM), which includes smoothing splines as a special case. The argument `bs="cr"` specifies the use of cubic regression splines.

Note that the *smooth.spline* function in Section 2.2, unlike GAM, fits a smoothing spline by optimizing the smoothing parameter using generalized cross-validation (or other criteria). However, one can attempt to make the splines fitted in Sections 2.2 and 2.3 as close as possible by setting the basis dimension `s(..., k)` and degrees of freedom `smooth.spline(..., df)` manually, i.e., $k = df + 1$ for cubic splines.

```
gam_fit <- mgcv::gam(s13$LYMP ~ s(s13$Time, bs="cr"), method="REML")

par(mar=c(5,5,5,5))
plot(gam_fit, main="Smoothing Spline Fit for IF-013",
     xlab="Month", ylab=expression(paste("Lymphocyte Count (\u00d7",10^9/L,")")))
points(s13$Time, s13$LYMP-mean(s13$LYMP), col="red") #Add the original data points to the fit
```



3. Summary

Fit a smoothing spline to each subject's measurement using the *gam* function.

Plot results along with the pointwise 95% confidence intervals. Estimate the first derivative (rate of change) and second derivative (concavity – rate of change in the slope) at Month 1; obtain 95% confidence intervals for both and annotate the plots with these two confidence intervals for each subject.

```
par(mfrow=c(1,2))

for (subj in unique(dat$ID)) {
  sub <- subset(dat, ID==subj)
  phases <- c(tail(subset(sub, Phase==0)[,"Time"],1), #start of fasting
              tail(subset(sub, Phase==1)[,"Time"],1)) #start of posttest

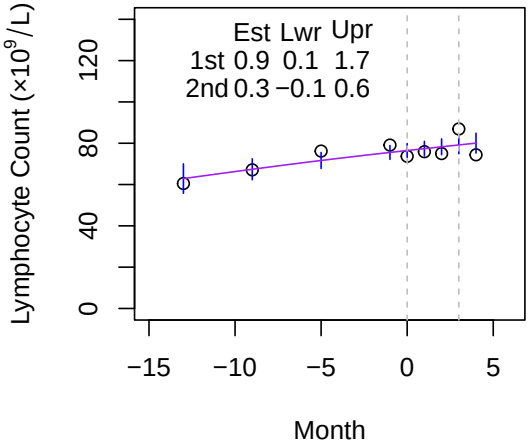
  fit <- mgcv::gam(LYMP ~ s(Time, bs="cr", k=7), data=sub, method="REML") #fit a GAM
  xvals <- seq(min(sub$Time), max(sub$Time), length.out=100)
  spline <- predict(fit, newdata=data.frame("Time"=xvals), type="response")

  pred <- predict(fit, type="response", se.fit=T) #get standard errors
  sub$fit <- pred$fit
  sub$se <- pred$se.fit
  sub$lwr <- sub$fit - 1.96 * sub$se
  sub$upr <- sub$fit + 1.96 * sub$se #compute 95% CIs

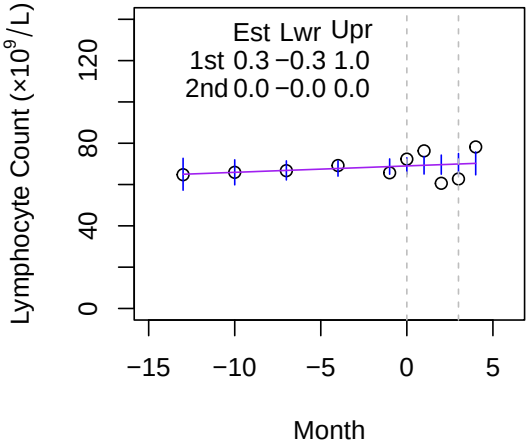
  par(mar=c(5,5,5,5))
  plot(sub$Time, sub$LYMP, ylim=c(0,140), xlim=c(-15,6),
       main=paste0("Spline Fit for ", subj),
       xlab="Month", ylab=expression(paste("Lymphocyte Count (\u00d7",10^9/L,")")))
  for(i in 1:nrow(sub)) { #add pointwise CIs
    lines(c(sub$Time[i], sub$Time[i]), c(sub$lwr[i], sub$upr[i]), col="blue")
  }
  lines(xvals, spline, col="purple")
  abline(v=phases, col="gray", lty=2)

  der1 <- gratia::derivatives(fit, data=data.frame("Time"=1)) #compute the first derivative
  der2 <- gratia::derivatives(fit, order=2, data=data.frame("Time"=1))
  tbl <- data.frame("Est"=c(der1$derivative, der2$derivative),
                    "Lwr"=c(der1$lower, der2$lower),
                    "Upr"=c(der1$upper, der2$upper))
  rownames(tbl) <- c("1st","2nd")
  tbl <- data.frame(apply(tbl, 2, function(x) formatC(x, format="f", digits=1)))
  plotrix::addtable2plot(x=-13, y=100, table=tbl, bty="n", display.rownames=T)
}
```

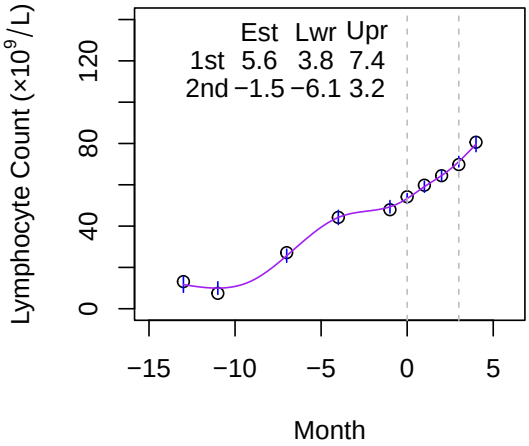
Spline Fit for IF−002



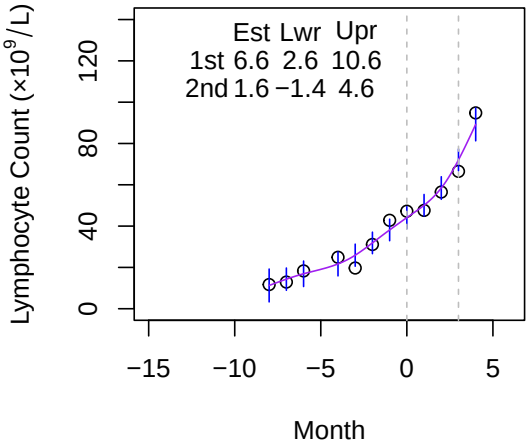
Spline Fit for IF−004



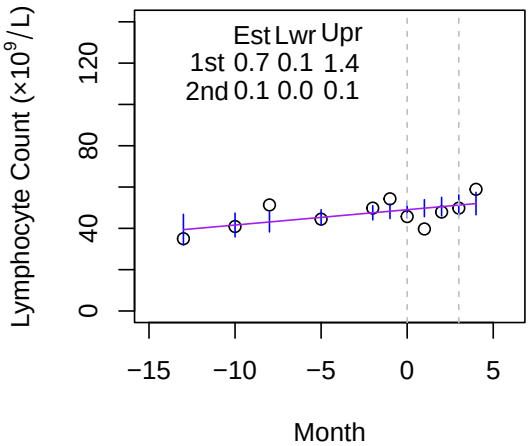
Spline Fit for IF−005



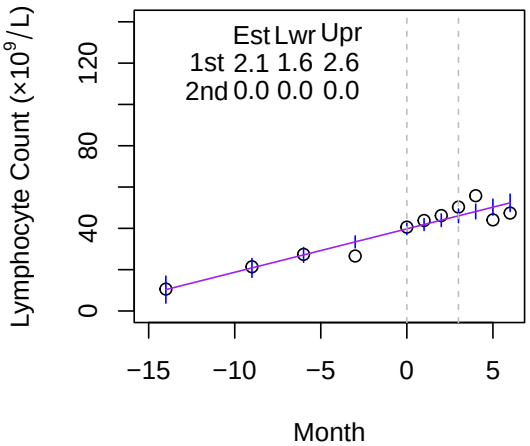
Spline Fit for IF−006

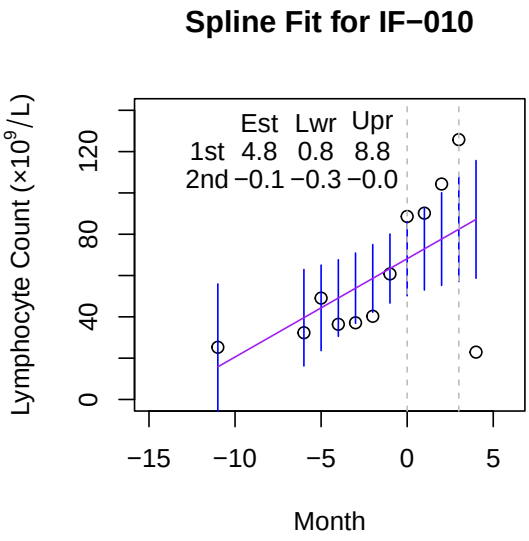


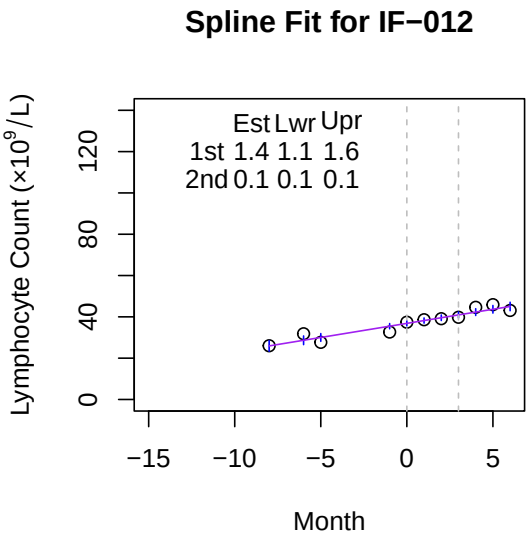
Spline Fit for IF−007

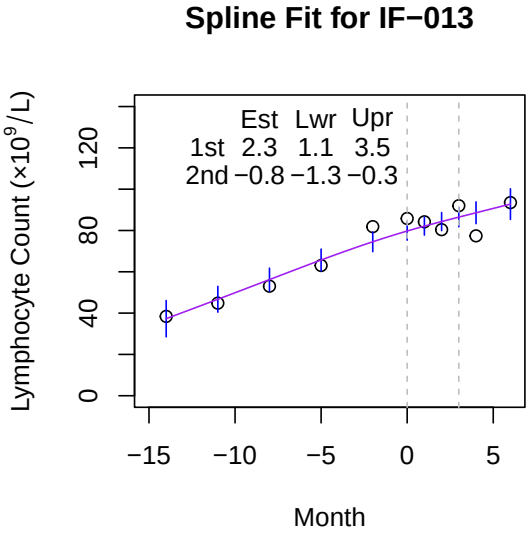


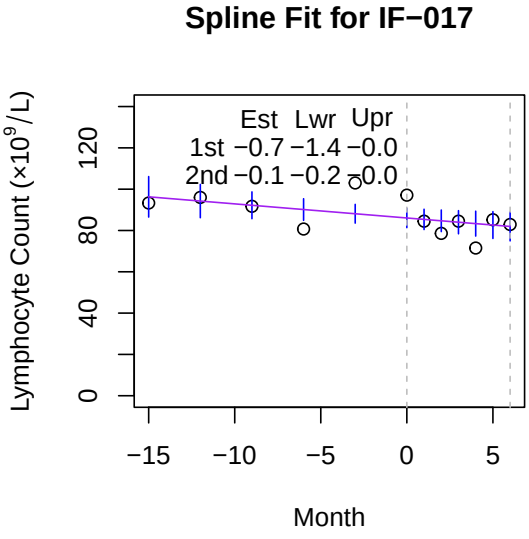
Spline Fit for IF−009

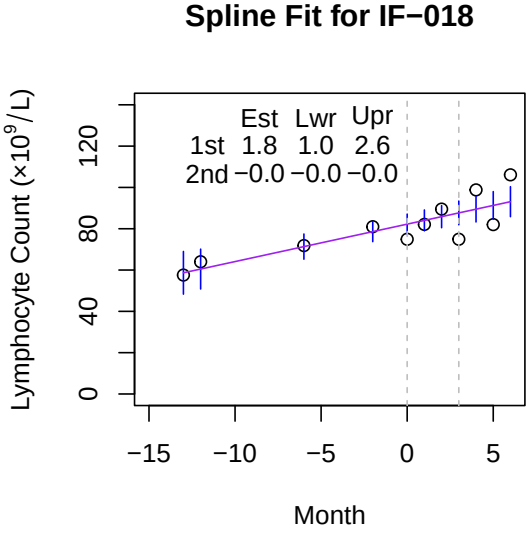


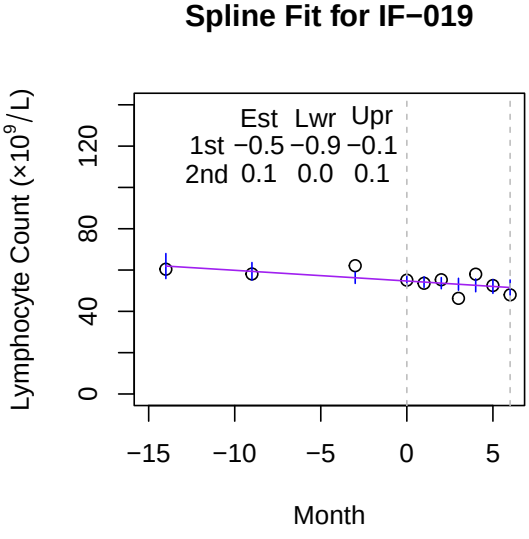

Spline Fit for IF−010
Est Lwr Upr
1st 4.8 0.8 8.8
2nd −0.1 −0.3 −0.0
Lymphocyte Count (×10^9/L)
0
40
80
120
−15
−10
−5
0
5
Month

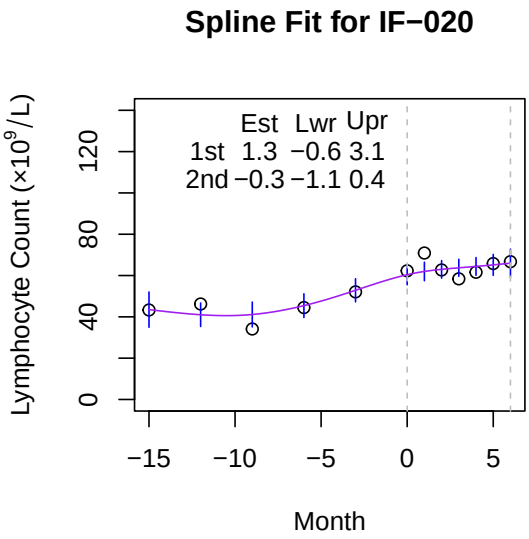

Spline Fit for IF−012
Est Lwr Upr
1st 1.4 1.1 1.6
2nd 0.1 0.1 0.1
Lymphocyte Count (×10^9/L)
0
40
80
120
−15
−10
−5
0
5
Month

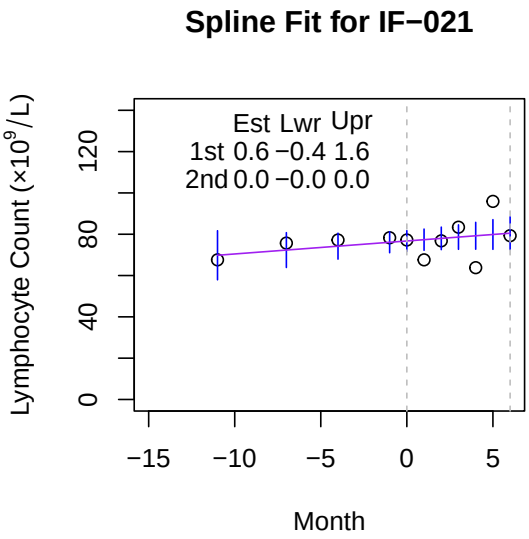

Spline Fit for IF−013
Est Lwr Upr
1st 2.3 1.1 3.5
2nd −0.8 −1.3 −0.3
Lymphocyte Count (×10^9/L)
0
40
80
120
−15
−10
−5
0
5
Month

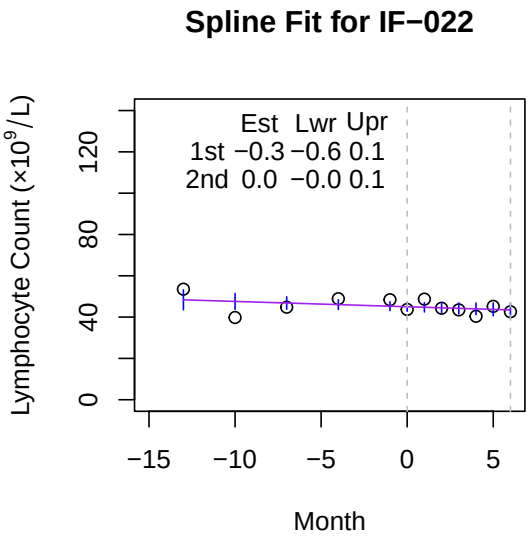

Spline Fit for IF−017
Est Lwr Upr
1st −0.7 −1.4 −0.0
2nd −0.1 −0.2 −0.0
Lymphocyte Count (×10^9/L)
0
40
80
120
−15
−10
−5
0
5
Month


Spline Fit for IF−018
Est Lwr Upr
1st 1.8 1.0 2.6
2nd −0.0 −0.0 −0.0
Lymphocyte Count (×10^9/L)
0
40
80
120
−15
−10
−5
0
5
Month


Spline Fit for IF−019
Est Lwr Upr
1st −0.5 −0.9 −0.1
2nd 0.1 0.0 0.1
Lymphocyte Count (×10^9/L)
0
40
80
120
−15
−10
−5
0
5
Month







References

Denis, M., Adriana, U., & Delphine, C. (2021). Estimation of the derivatives. https://cran.r-project.org/web/packages/doremi/vignettes/derivatives.html